

Hệ thống truyền nhận file qua Socket

01 · TỔNG QUAN

Mục tiêu & nguyên tắc thực hiện

Đồ án mô phỏng bài toán thực tế: xây dựng một dịch vụ chia sẻ file nội bộ. Xây dựng ứng dụng theo 2 giai đoạn: cơ bản, nâng cao. Mỗi giai đoạn là một cột mốc kỹ thuật riêng biệt, sinh viên kế thừa toàn bộ code của giai đoạn trước và mở rộng thêm — không được viết lại từ đầu.

Mục tiêu kỹ thuật

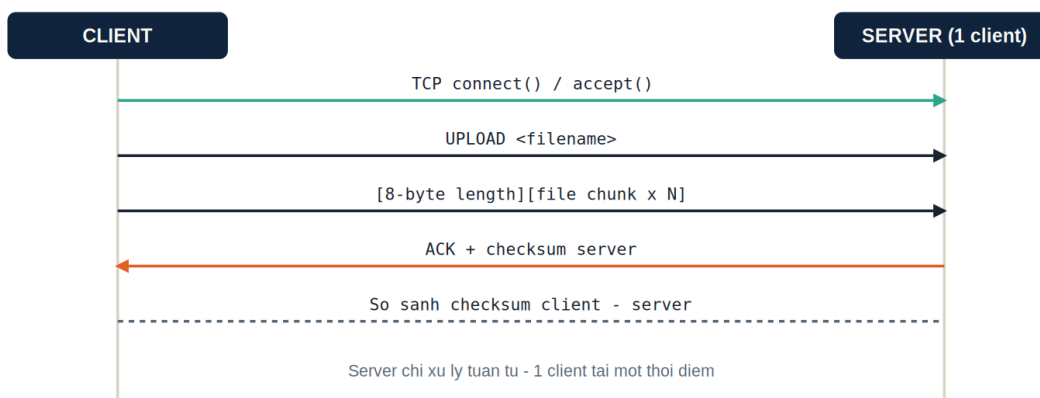
- **Socket API tầng TCP:** `socket()`, `bind()`, `listen()`, `accept()`, `connect()`, `send()/recv()`.
- **Message boundary:** hiểu và tự giải quyết vấn đề TCP không giữ ranh giới gói tin ứng dụng (giao thức stream).
- **Thiết kế giao thức:** xây dựng giao thức tầng ứng dụng có đặc tả rõ ràng, tương tự cách các RFC mô tả giao thức thật.
- **I/O đồng thời:** làm việc với concurrency / multiplexing để phục vụ nhiều client cùng lúc.
- **Chịu lỗi:** xử lý các tình huống gián đoạn mạng như hệ thống thực tế — không giả định mạng luôn hoàn hảo.

02 · KIẾN TRÚC HỆ THỐNG

Sơ đồ kiến trúc & luồng dữ liệu

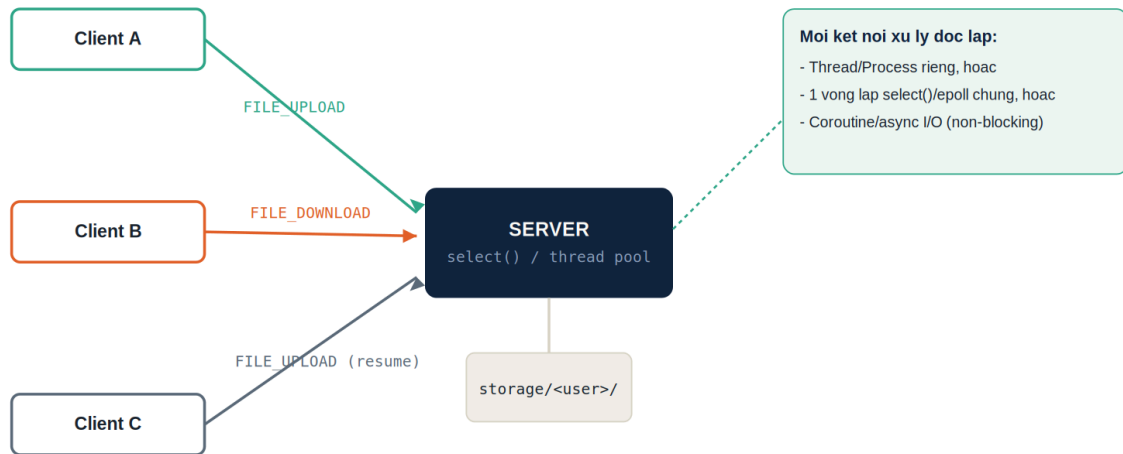
Hai sơ đồ dưới đây mô tả sự khác biệt cốt lõi giữa hai giai đoạn: từ một kết nối đơn tuần tự, sang một server phục vụ nhiều client song song với giao thức đóng gói riêng.

Giai đoạn 1 — Client-Server đơn luồng



Hình 1 — Trình tự trao đổi thông điệp trong Giai đoạn 1 (sequence diagram)

Giai đoạn 2 — Đa client với giao thức tùy chỉnh



Hình 2 — Kiến trúc đa client Giai đoạn 2: mỗi user có namespace lưu trữ riêng

Giai đoạn 1 — File Transfer đơn luồng · 4,0 điểm

Xây dựng một cặp chương trình client/server dùng TCP: server chỉ phục vụ một client tại một thời điểm, nhưng phải truyền file đúng, toàn vẹn, và có khả năng chịu lỗi cơ bản.

Chức năng bắt buộc

- Server lắng nghe trên một cổng TCP cố định (cấu hình được qua tham số dòng lệnh hoặc file config).
- Client hỗ trợ đúng 3 lệnh: LIST, UPLOAD <filename>, DOWNLOAD <filename>.
- Dữ liệu file truyền theo từng chunk cố định (khuyến nghị 4KB–64KB/lần), không đọc toàn bộ file vào bộ nhớ.
- Sau khi truyền xong, hai bên tính và so sánh checksum (MD5 hoặc SHA-256) để xác nhận toàn vẹn.
- Server ghi log mỗi phiên: thời điểm kết nối, lệnh thực thi, thời gian truyền, tốc độ trung bình (KB/s).

Message boundary trên TCP: TCP là giao thức stream, không tự giữ ranh giới giữa các lần gọi send(). Sinh viên phải tự thiết kế cơ chế đóng khung (framing) — ví dụ gửi trước 8 byte biểu diễn độ dài nội dung, hoặc dùng ký tự phân tách — để bên nhận biết chính xác khi nào một lệnh hoặc một file đã truyền xong.

Bảng yêu cầu & điểm chi tiết — Giai đoạn 1

R1.1 — Thiết lập kết nối & giao thức lệnh cơ bản [1,0 điểm]

TCP handshake · LIST/UPLOAD/DOWNLOAD

Tiêu chí chấm:

- Kết nối TCP đúng, xử lý được khi client kết nối/ngắt nhiều lần liên tiếp (0,4đ)
- Cả 3 lệnh LIST / UPLOAD / DOWNLOAD hoạt động đúng chức năng (0,4đ)
- Phản hồi lỗi hợp lý khi lệnh không hợp lệ hoặc thiếu tham số (0,2đ)

R1.2 — Truyền file đúng & xử lý message boundary [1,0 điểm]

Framing · chunking

Tiêu chí chấm:

- Truyền đúng file nhỏ (<1KB), trung bình (~10MB), lớn (>100MB) (0,5đ)
- Có cơ chế framing rõ ràng (length-prefix hoặc delimiter), giải thích trong báo cáo (0,3đ)
- Không load toàn bộ file vào RAM — xác minh qua theo dõi bộ nhớ khi test file lớn (0,2đ)

R1.3 — Toàn vẹn dữ liệu bằng checksum [0,5 điểm]

MD5 / SHA-256

Tiêu chí chấm:

- Checksum tính ở cả client và server, khớp 100% với mọi file test (0,3đ)
- Cảnh báo rõ ràng khi checksum không khớp (giả lập bằng cách làm hỏng file cố ý) (0,2đ)

R1.4 — Ghi log hoạt động server [0,5 điểm]

Session log

Tiêu chí chấm:

- Log đầy đủ: thời điểm, IP client, lệnh, thời gian xử lý, tốc độ truyền **(0,3đ)**
- Log có định dạng nhất quán, dễ đọc hoặc dễ parse **(0,2đ)**

R1.5 — Xử lý lỗi cơ bản, không crash server [0,5 điểm]

Error handling

Tiêu chí chấm:

- File không tồn tại, tên file rỗng/không hợp lệ được xử lý gọn gàng **(0,2đ)**
- Client ngắt kết nối giữa chừng không làm server treo hoặc thoát chương trình **(0,3đ)**

R1.6 — Báo cáo kỹ thuật Giai đoạn 1 [0,5 điểm]

Report

Tiêu chí chấm:

- Giải thích rõ cơ chế xử lý message boundary đã chọn và lý do **(0,3đ)**
- Có kết quả đo tốc độ truyền thực tế với ít nhất 3 kích thước file khác nhau **(0,2đ)**

Giai đoạn 2 — Đa Client & Truyền File Nâng Cao · 6,0 điểm

Nâng cấp toàn bộ hệ thống Giai đoạn 1 để phục vụ nhiều client đồng thời, dựa trên một giao thức tầng ứng dụng tự thiết kế có cấu trúc nhị phân rõ ràng, hỗ trợ resume transfer và kiểm soát tài nguyên.

Chức năng bắt buộc

- **Đa client đồng thời** — chọn một trong ba kỹ thuật: thread/process-per-client, I/O multiplexing (select/poll/epoll), hoặc async I/O. Nêu rõ lý do lựa chọn trong báo cáo.
- **Giao thức tùy chỉnh** theo cấu trúc Length–Opcode–Payload (chi tiết ở mục Đặc tả giao thức).
- **Đăng nhập bằng username** duy nhất (chưa yêu cầu mật khẩu); mỗi user có thư mục lưu trữ riêng trên server.
- **Resume transfer**: nếu kết nối đứt giữa chừng, client tiếp tục truyền từ vị trí byte đã dừng (offset-based), không truyền lại từ đầu.
- **Progress tracking**: hiển thị phần trăm hoàn thành theo thời gian thực khi truyền.
- **Giới hạn băng thông (throttling)**: mỗi client bị giới hạn tốc độ tối đa (ví dụ 500KB/s), cấu hình được.

Bảng yêu cầu & điểm chi tiết — Giai đoạn 2

R2.1 — Xử lý đa client đồng thời, ổn định [1,5 điểm]

Concurrency

Tiêu chí chấm:

- Phục vụ ổn định tối thiểu 10 client đồng thời, có script giả lập kiểm chứng (0,6đ)
- Không xảy ra deadlock, race condition khi nhiều client thao tác cùng lúc (0,5đ)
- Giải phóng tài nguyên đúng khi client ngắt kết nối liên tục (connect/disconnect lặp lại) (0,4đ)

R2.2 — Thiết kế & đặc tả giao thức tùy chỉnh [1,0 điểm]

Protocol spec (mini-RFC)

Tiêu chí chấm:

- Cấu trúc gói tin Length–Opcode–Payload triển khai đúng, nhất quán ở cả client và server (0,5đ)
- Tài liệu đặc tả đầy đủ: bảng opcode, định dạng payload từng loại, ví dụ hex dump (0,5đ)

R2.3 — Namespace riêng & thao tác song song không xung đột [1,0 điểm]

Per-user storage

Tiêu chí chấm:

- Mỗi username có thư mục riêng, không truy cập chéo file của user khác (0,5đ)
- Nhiều client upload/download các file khác nhau cùng lúc không gây hỏng dữ liệu (0,5đ)

R2.4 — Resume transfer (tiếp tục truyền từ điểm dừng) [1,0 điểm]

Offset-based resume

Tiêu chí chấm:

- Ngắt kết nối giữa chừng khi truyền file lớn, reconnect và tiếp tục đúng từ offset đã dừng **(0,7đ)**
- File sau khi resume vẫn cho checksum khớp hoàn toàn với bản gốc **(0,3đ)**

R2.5 — Progress tracking thời gian thực [0,5 điểm]

% completion

Tiêu chí chấm:

- Hiển thị % tiến trình cập nhật liên tục, sai số chấp nhận được so với thực tế **(0,5đ)**

R2.6 — Giới hạn băng thông theo client [0,5 điểm]

Bandwidth throttling

Tiêu chí chấm:

- Tốc độ truyền thực đo được không vượt quá ngưỡng cấu hình quá 10% **(0,5đ)**

R2.7 — Xử lý tình huống thực tế [0,5 điểm]

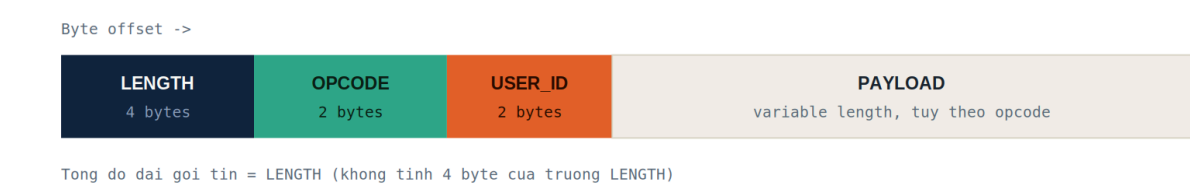
Edge cases

Tiêu chí chấm:

- Trùng tên file khi upload xử lý theo chính sách rõ ràng, nêu trong báo cáo **(0,2đ)**
- Vượt số client tối đa: server từ chối kết nối mới kèm mã lỗi hợp lệ, không crash **(0,3đ)**

Đặc tả giao thức tùy chỉnh

Cấu trúc gói tin đề xuất cho Giai đoạn 2 — sinh viên có thể điều chỉnh chi tiết nhưng phải giữ nguyên tắc length-prefixed framing để tránh vấn đề message boundary đã gặp ở Giai đoạn 1.



Hình 3 — Cấu trúc khung gói tin (packet framing) đề xuất

Bảng Opcode gợi ý

Opcode	Hex	Chiều	Payload
LOGIN	0x01	Client → Server	username (UTF-8, độ dài biến đổi)
LOGOUT	0x02	Client → Server	rỗng
FILE_LIST	0x10	Client → Server	rỗng
FILE_LIST_RESP	0x11	Server → Client	danh sách tên file + kích thước
FILE_UPLOAD	0x12	Client → Server	tên file, tổng kích thước, offset, chunk
FILE_DOWNLOAD	0x13	Client → Server	tên file, offset yêu cầu tiếp tục
FILE_CHUNK	0x14	Server ↔ Client	offset hiện tại, dữ liệu nhị phân
FILE_DELETE	0x15	Client → Server	tên file
ACK	0x20	Server → Client	opcode được xác nhận
ERROR	0xFF	Server → Client	mã lỗi + thông điệp text

Ví dụ đóng gói yêu cầu resume download

```
LENGTH = 0x00000019 // 25 bytes payload phía sau
OPCODE = 0x13 // FILE_DOWNLOAD
USER_ID = 0x0007
PAYLOAD = "report.pdf" + OFFSET(8 bytes = 4194304)
// yêu cầu tải tiếp từ byte thứ 4.194.304
```

Lưu ý khi thiết kế: Vì gói tin có thể chứa dữ liệu file (binary), payload phải được xử lý ở mức byte thô — tuyệt đối không dùng các hàm xử lý chuỗi có thể dừng sớm khi gặp byte 0x00 (lỗi thường gặp khi dùng chuỗi C-style hoặc strlen() trên buffer nhị phân).

Ý tưởng chia gói

Để không phải lặp lại tên file trong mỗi gói, upload được tách làm 2 loại gói:

- **FILE_UPLOAD (0x12)** — gói *mở đầu*, mang metadata: tên file, tổng kích thước, offset bắt đầu.
- **FILE_CHUNK (0x14)** — các gói *dữ liệu*, chỉ mang **offset + data**. Nhờ **offset** đi kèm mỗi chunk, server ghi đúng vị trí và đây cũng là chìa khóa để resume.

Cấu trúc payload của **FILE_CHUNK**:

Trường	Kích thước	Ý nghĩa
offset	8 bytes	vị trí byte đầu tiên của chunk này trong file
data	biến đổi	dữ liệu nhị phân thô của chunk

Ví dụ cụ thể

Upload file **data.bin**, tổng **10.000.000 byte**, chunk size **4 KB (4096 byte)**, **USER_ID = 0x0007**.

Bước 1 — Client gửi gói mở đầu (FILE_UPLOAD):

```

LENGTH = 0x0000001E          // 30 bytes phía sau
OPCODE = 0x12                  // FILE_UPLOAD
USER_ID = 0x0007
PAYLOAD:
  filename_len = 0x0008        // "data.bin" = 8 byte
  filename     = "data.bin"
  total_size   = 0x000000000989680 // 10,000,000
  offset       = 0x0000000000000000 // gửi từ đầu
// payload = 2 + 8 + 8 + 8 = 26 -> LENGTH = 2(opcode)+2(user)+26 = 30

```

Bước 2 — Server xác nhận, báo offset cần nhận (ACK):

```

LENGTH = 0x0000000E          // 14 bytes
OPCODE = 0x20                  // ACK
USER_ID = 0x0007
PAYLOAD:
  acked_opcode = 0x0012
  next_offset  = 0x0000000000000000 // server chưa có byte nào

```

Bước 3 — Client gửi từng FILE_CHUNK 4 KB:

```

LENGTH = 0x0000100C          // 4108 bytes
OPCODE = 0x14                  // FILE_CHUNK
USER_ID = 0x0007
PAYLOAD:
  offset = 0x0000000000000000
  data   = <4096 byte nhị phân>
// payload = 8 + 4096 = 4104 -> LENGTH = 2 + 2 + 4104 = 4108

```

--- chunk kế tiếp ---


```
offset = 0x0000000000001000 // 4096
data = <4096 byte>
--- chunk kế tiếp ---
offset = 0x0000000000002000 // 8192
data = <4096 byte>
... lặp lại ...
```

Bước 4 — Chunk cuối (dư, chỉ 1664 byte):

```
LENGTH = 0x0000068C // 1676 bytes
OPCODE = 0x14 // FILE_CHUNK
USER_ID = 0x0007
PAYLOAD:
offset = 0x0000000000989000 // 9,998,336 (= 2441 × 4096)
data = <1664 byte>
// 10,000,000 - 9,998,336 = 1664 byte = 0x680
// payload = 8 + 1664 = 1672 -> LENGTH = 2 + 2 + 1672 = 1676
```

Bước 5 — Server xác nhận đã nhận đủ:

```
LENGTH = 0x0000000E
OPCODE = 0x20 // ACK
USER_ID = 0x0007
PAYLOAD:
acked_opcode = 0x0014
next_offset = 0x0000000000989680 // = total_size => hoàn tất
```

Trường hợp resume (đứt kết nối giữa chừng)

Giả sử kết nối rớt sau khi server đã nhận **6.000.000 byte**. Client kết nối lại, gửi lại gói **FILE_UPLOAD** mở đầu (giống Bước 1). Server kiểm tra file dở dang và trả về offset đã có; client **chỉ gửi tiếp phần còn thiếu**, không truyền lại 6 MB đầu:

```
// Server trả offset đã nhận:
LENGTH = 0x0000000E
OPCODE = 0x20 // ACK
USER_ID = 0x0007
PAYLOAD:
acked_opcode = 0x0012
next_offset = 0x00000000005B8D80 // 6,000,000 byte đã có
```

```
// Client tiếp tục từ offset 6,000,000:
LENGTH = 0x0000100C
OPCODE = 0x14 // FILE_CHUNK
USER_ID = 0x0007
PAYLOAD:
offset = 0x00000000005B8D80
data = <4096 byte>
```

Ba điểm mấu chốt sinh viên cần lưu ý: **offset** phải đi kèm **mọi** chunk (không dựa vào thứ tự đến, vì có thể ghi lại sau resume); trường **data** là byte thô nên tuyệt đối không xử lý bằng hàm chuỗi dừng ở byte **0x00**; và server nên chốt bằng ACK cuối có **next_offset == total_size** để cả hai bên biết đã truyền xong trước khi so khớp checksum.

PHỤ LỤC KỸ THUẬT

Ngôn ngữ & thư viện gợi ý

Sinh viên được tự do chọn 1 trong 5 ngôn ngữ. Điểm số không phụ thuộc vào ngôn ngữ chọn, miễn đáp ứng đầy đủ tiêu chí.

Ngôn ngữ	Thư viện chính	Ghi chú
Python	socket, selectors, hashlib, threading	Nhanh để phát triển; dùng selectors cho multiplexing hoặc asyncio cho async I/O.
C	sys/socket.h, poll/epoll, pthread, openssl	Sát socket API gốc, hiểu rõ byte-level framing; tự quản lý bộ nhớ cẩn thận.
C++	Boost.Asio, std::thread, OpenSSL	Boost.Asio hỗ trợ tốt cả blocking và async, hợp cho Giai đoạn 2.
Java	java.nio, Selector, ExecutorService	java.nio Selector ánh xạ trực tiếp mô hình I/O multiplexing.
C#	System.Net.Sockets, async/await, Cryptography	SocketAsyncEventArgs hoặc async/await hợp mô hình async hiện đại.

ĐÁNH GIÁ

Bảng tổng điểm

Mã	Yêu cầu	GĐ	Điểm
R1.1	Kết nối & giao thức lệnh cơ bản	1	1,0
R1.2	Truyền file & xử lý message boundary	1	1,0
R1.3	Toàn vẹn dữ liệu (checksum)	1	0,5
R1.4	Ghi log hoạt động server	1	0,5
R1.5	Xử lý lỗi cơ bản	1	0,5
R1.6	Báo cáo kỹ thuật Giai đoạn 1	1	0,5
R2.1	Đa client đồng thời, ổn định	2	1,5
R2.2	Thiết kế & đặc tả giao thức tùy chỉnh	2	1,0
R2.3	Namespace riêng, song song không xung đột	2	1,0
R2.4	Resume transfer	2	1,0
R2.5	Progress tracking	2	0,5
R2.6	Giới hạn băng thông	2	0,5
R2.7	Xử lý tình huống thực tế	2	0,5
	TỔNG CỘNG		10,0

QUY ĐỊNH

Quy định nộp bài

- **Đồ án nhóm 3 sinh viên.**
- **Mã nguồn:** nộp dưới dạng file zip.
- **Báo cáo:** file PDF hoặc Word gồm mô tả kiến trúc, đặc tả giao thức, kết quả kiểm thử (số liệu tốc độ, ảnh chụp log) và giải thích các lựa chọn kỹ thuật.
- **Demo:** demo trực tiếp hoặc video ngắn (5–8 phút): truyền file cơ bản, tối thiểu 3 client đồng thời, và ít nhất một lần resume transfer thành công sau khi ngắt mạng giữa chừng.
- **Điều kiện tối thiểu:** không đạt chức năng LIST/UPLOAD/DOWNLOAD cơ bản ở Giai đoạn 1 sẽ không được chấm điểm Giai đoạn 2, kể cả khi đã cài đặt.